Công nghệ Chuyển đổi số

(Digital Transformation Technologies)

Các công nghệ số trong chuyển đổi số

PGS. TS. TSKH. Hoàng Đăng Hải

Hdhai - Copyright

Hà nội, 2026

1

Nội dung chính trong môn học

I- Tổng quan về chuyển đổi số
- Các khái niệm và vấn đề liên quan

II- Các nhóm công nghệ số trong CĐS
- Các công nghệ số điển hình trong CĐS

III- Các nhóm công nghệ nền tảng của CĐS
- Các công nghệ nền tảng CĐS: Cloud, Big Data, NGN, IoT, ML+AI, Security

IV- Triển khai chuyển đổi số
- Mô hình, kiến trúc, chiến lược chuyển đổi số
- Triển khai CĐS trong tổ chức, doanh nghiệp

Hdhai - Copyright

2

2.6. Thu thập nguồn dữ liệu từ máy quét

Từ bản giấy → Làm thế nào ?
Bản giấy: chữ in, viết tay, ảnh vẽ, ảnh chụp

- Quét (Scan) bản giấy → ảnh / PDF
- Chụp bản giấy → ảnh → …
- Công cụ số hóa (=Scanner)

1. Tiền xử lý ảnh:
   → làm sạch, giảm nhiễu, tăng chất lượng (độ nét, độ sáng)
2. Nhận dạng chữ viết
   → So khớp mẫu
   → Tách đặc trưng
   → Phân tích, hiệu chỉnh, sửa lỗi

Hdhai - Copyright

3

3

2.6. Thu thập nguồn dữ liệu từ máy quét

Ví dụ: Quét mẫu dữ liệu từ máy quét OCR (Optical Character Recognition)

Bước tiền xử lý OCR

- Xoay ảnh về vị trí đúng (hướng, mặt ngang) –dựa vào tính góc
- Cắt ảnh – chỉ lấy phần quan trọng (xác định vùng quan tâm)
- Phóng ảnh theo kích thước và độ phân giải đúng với mẫu
- Chuyển ảnh đen trắng –giảm độ phức tạp
- Hiệu chỉnh Skew (giảm độ méo ảnh)
- Loại nhiễu: các vết bẩn hoặc vết loang màu
- Tạo shape: giảm mỏng nét chữ thành 1 điểm ảnh, tạo xương chữ

Hdhai - Copyright

4

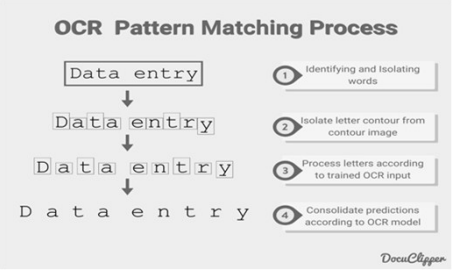
4

2.6. Thu thập nguồn dữ liệu từ máy quét

Ví dụ: Quét mẫu dữ liệu từ máy quét OCR (Optical Character Recognition)

Tiến trình nhận dạng ký tự

- Huấn luyện mẫu ký tự (các font và kiểu)
- Tách các biểu tượng ký tự từ ảnh
- So sánh biểu tượng với mẫu trong CSDL
- Xác định ký tự theo mô hình đã luyện



Ánh xạ ký tự

- Mỗi ký tự đã nhận dạng từ ảnh được chuyển sang dạng số
- Ánh xạ từ tập mẫu ký tự để chuyển đổi và lưu mã máy cho ký tự đã nhận dạng

Hdhai - Copyright

5

5

Bài tập ví dụ: Thuật toán Levenshtein

Thuật toán Levenshtein: so sánh sự khác biệt giữa hai chuỗi ký tự

Đo sự khác biệt giữa hai chuỗi ký tự a và b = số ký tự tối thiểu cần thay đổi (thêm, bớt, thay thế) để biến đổi chuỗi ký tự a thành chuỗi ký tự b.

Khoảng cách biệt càng nhỏ → 2 chuỗi càng giống nhau !

Hai chuỗi ký tự a và b có chiều dài là |a| và |b|

Khoảng cách biệt *lev(a,b)*:

$$lev(a,b) = \begin{cases} |a| & if\ |b| = 0 \\ |b| & if\ |a| = 0 \\ lev(tail(a), tail(b)) & if\ head(a) = head(b) \\ \min \begin{Bmatrix} lev(tail(a), b) & +1 \\ lev(a, tail(b) & +1 \\ lev(tail(a), tail(b) & +1 \end{Bmatrix} & otherwise \end{cases}$$

tail($x_0\ x_1 \dots x_n$) = ($x_1 \dots x_n$) *head*($x_0\ x_1 \dots x_n$) =(x_0) = x(0)

Hdhai - Copyright

6

6

Bài tập ví dụ: Thuật toán Levenshtein

Thuật toán Levenshtein: so sánh sự khác biệt giữa hai chuỗi ký tự

Đo sự khác biệt giữa hai chuỗi ký tự a và b = số ký tự tối thiểu cần thay đổi (thêm, bớt, thay thế) để biến đổi chuỗi ký tự a thành chuỗi ký tự b.

Khoảng cách biệt càng nhỏ → 2 chuỗi càng giống nhau !

Hai chuỗi ký tự a và b có chiều dài là |a| và |b|

Khoảng cách biệt *lev(a,b)*:

$$lev(a,b) = \begin{cases} |a| & if\ |b| = 0 \\ |b| & if\ |a| = 0 \\ lev(tail(a), tail(b) & if\ head(a) = head(b) \\ \min \begin{Bmatrix} lev(tail(a), b) & +1 \\ lev(a, tail(b) & +1 \\ lev(tail(a), tail(b) & +1 \end{Bmatrix} & otherwise \end{cases}$$

tail($x_0\ x_1 \dots x_n$) = ($x_1 \dots x_n$) *head($x_0\ x_1 \dots x_n$) =(x_0) = x(0)*

1) $lev(i,j) = |a| \quad if\ |b| = 0$

2) $lev(i,j) = |b| \quad if\ |a| = 0$

3) $lev(i,j) = min \begin{Bmatrix} lev(i, j-1) & +1 \\ lev(i-1, j-1) & +1 \quad if \quad (ai \neq bj)\ ;\ +0 \quad if\ \ (ai = bj) \\ lev(i-1, j) & +1 \end{Bmatrix}$

Hdhai - Copyright

7

7

2.7. Thu thập dữ liệu ảnh số

Định dạng ảnh (Formats)

- **Loại ảnh:**

1) Dạng lưới (Raster, Bitmap): Dựa trên Pixel (BMP, JPG, JPEG, PNG, GIF, TIFF,…)
2) Dạng Vector: Không dựa trên Pixel mà dùng công thức toán học để tập hợp các đường thẳng, cong, tròn… (EPS, PDF, SVG scalable vector graphics)

Khi phóng ảnh sẽ phát hiện sự khác biệt.

- **Màu ảnh:** R (Red), G (Green), Blue (B). Cường độ sáng I (Intensity)

- **Định dạng ảnh phổ biến:** BMP, GIF, JPEG (jpg, jpeg), PNG, TIFF, JPEG2000…

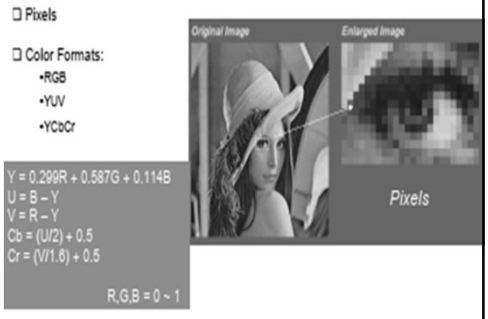
Hdhai - Copyright

8

Cấu trúc dữ liệu ảnh số

Cấu trúc ảnh số:

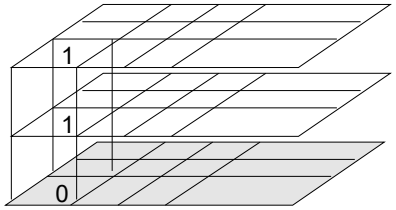
- Mỗi ảnh được lưu thành ma trận M x N
- Mỗi điểm của ma trận tương ứng 1 vị trí điểm ảnh
- Ảnh đen trắng: mỗi điểm = 8 bit (giá trị từ 0….255)
- Ảnh màu: mỗi điểm chứa RGB x 8 bit (3 x 8 =24 bit = 16 triệu màu)



Cấu trúc Bit Plane biểu thị mức độ sáng tối (Grayscale) :

78	100	100	100	100	100	100	100
100	**78**	0	0	0	0	0	100
100	0	**78**	0	0	0	0	100
100	0	0	**78**	0	0	0	100
100	0	0	0	**78**	0	0	100
100	0	0	0	0	**78**	0	100
100	0	0	0	0	0	**78**	100
100	100	100	100	100	100	100	**78**

78 = 0 1 0 0 1 1 1 0



9

Các loại định dạng ảnh (Formats)

1) BMP (Bitmap / Microsoft)

- Tập điểm ảnh. Mỗi điểm ảnh là 1 khối: 1, 4, 8, 16, 24, 32 bits
- Không nén → chất lượng cao, kích thước lớn. Nén với Run-length Coding (RLC)

2) GIF (Graphics Interchanged Format) / PCX

- Tập điểm ảnh. Mỗi điểm ảnh là 1 khối: 1…8 bits → Tối đa 256 màu
- Kích thước nhỏ, thích hợp cho Web. Nén với LZW (lossless) hoặc Color quantization (lossy)

3) JPEG (Joint Photographic Expert Groups) (JPG)

- Ảnh lưu 24 bits/điểm ảnh (True Color) dạng chuỗi mã với hệ số DCT lượng tử hóa.
- Nén với DCT, kết hợp RLC+Huffman. Tỷ lệ chất lượng nén 10:1

4) PNG (Portable Network Graphics)

- Ảnh lưu 8 / 24 bits/điểm ảnh (True Color).
- Nén với Color quantization (lossy) hay Deflat (lossless) (LZ+Huffman)

5) TIFF (Tagged Image File Format)

- Ảnh lưu 8 / 16 bits/màu, chia theo các trường thẻ/giá trị.
- Không nén: kích thước lớn, chất lượng cao
- Nén (lossless) với RLC, Deflat (LZ+Huffman), LZW

6) JPEG2000 (JP2)

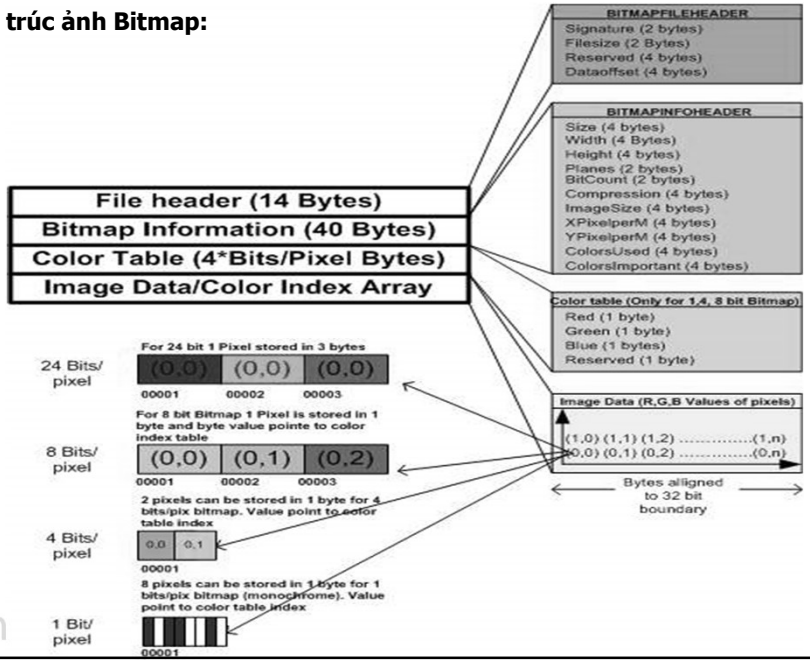
- Cải tiến JPEG. Nén với Discrete Wavelet Transform (DWT)

Hdhat - Copyright

10

Định dạng ảnh số BMP

Cấu trúc ảnh Bitmap:



Hdh

11

Cấu trúc ảnh số BMP

Bitmap Header

```
Type BITMAPFILEHEADER
  bfType As Integer      //ASCII string 'BM', giá trị hex của 'BM' là     '4D42' (thứ tự ghi trong bộ nhớ)
  bfSize As Long         //kích thước của toàn bộ file tính bằng byte
  bfReserved1 As Integer        // Chưa dùng
  bfReserved2 As Integer        // Chưa dùng
  bfOffBits As Long             // Vị trí bắt đầu dữ liệu ảnh
End Type
```

Bitmap Info Header:

```
Type BITMAPINFOHEADER
  biSize As Long
  biWidth As Long
  biHeight As Long
  biPlanes As Integer
  biBitCount As Integer
  biCompression As Long
  biSizeImage As Long
  biXPelsPerMeter As Long
  biYPelsPerMeter As Long
  biClrUsed As Long
  biClrImportant As Long
End Type
```

biSize : Kích thước Info Heade = 40 bytes.
biWidth : Độ rộng ảnh (pixels).
biHeight : Độ cao ảnh (pixels).
biPlanes : Số Planes (thường =1)
biBitCount : Số bit mã hóa màu, = 1 (BW),4 (16 màu),8 (256 màu),16 (16 tr.màu)
biCompression : Kiểu nén (RLE, BI RGB,…)
biSizeImage : Độ dài dữ liệu ảnh bitmap (số Pixel thực tế) in Bytes
biXPelsPerMeter, biYPelsPerMeter : ---
biClrUsed : ---
biClrImportant : ---

Hdhai - Copyright

12

Ví dụ: Cấu trúc ảnh số BMP

Cấu trúc ảnh Bitmap

- Mỗi ảnh được lưu thành ma trận M x N
- Mỗi điểm của ma trận tương ứng 1 vị trí điểm ảnh
- Ảnh đen trắng: mỗi điểm = 8 bit (giá trị từ 0….255)
- Ảnh màu: mỗi điểm chứa RGB x 8 bit (3x8=24 bit=16 tr màu)

```
3 3 3 3 3 3 3 3
0 1 4 1 4 1 4 0
0 4 1 4 1 4 1 0
0 5 5 5 5 5 5 0
0 5 5 5 5 5 5 0
0 1 4 1 4 1 4 0
0 4 1 4 1 4 1 0
2 2 2 2 2 2 2 2
```

0	000000
1	FF0000
2	00FF00
3	0000FF
4	FFFFFF
5	FFFF00
6	FF00FF
7	00FFFF
8	FF0080
9	FF8040
A	804000
B	008080
C	800000
D	800080
E	8080FF

Trên hình, mỗi pixel được biểu diễn bằng một số 4 bit → có 2^4 = 16 màu trong bảng màu.

Mỗi màu trong bảng được biểu diễn bằng một số 24 bit:

8 bit Red, 8 bit Green, 8 bit Blue.

Các số được hiển thị ở dạng Hex (cơ số 16):

A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

Hdhai - Copyright

13

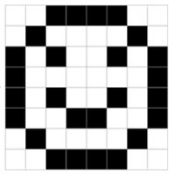
Ví dụ: Cấu trúc ảnh số BMP

Cấu trúc ảnh Bitmap

VD ảnh đen trắng, mỗi pixel được biểu diễn bằng 1 bit → có 2^1 = 2 màu (B & W).

Biểu thị B = 0, W = 1. Nếu 8 bit → 256 mức xám

```
1 1 0 0 0 0 1 1
1 0 1 1 1 1 0 1
0 1 0 1 1 0 1 0
0 1 1 1 1 1 1 0
0 1 0 1 1 0 1 0
0 1 1 0 0 1 1 0
1 0 1 1 1 1 0 1
1 1 0 0 0 0 1 1
```



Color intensity	DEC	HEX	BIN
	0	0x00	00000000
	16	0x10	00010000
	32	0x20	00100000
	48	0x30	00110000
	64	0x40	01000000
	80	0x50	01010000
	96	0x60	01100000
	112	0x70	01110000
	128	0x80	10000000
	144	0x90	10010000
	160	0xA0	10100000
	176	0xB0	10110000
	192	0xC0	11000000
	208	0xD0	11010000
	224	0xE0	11100000
	255	0xFF	11111111

Nén ảnh: RLE (Run Length Encoding): để giảm kích thước ảnh

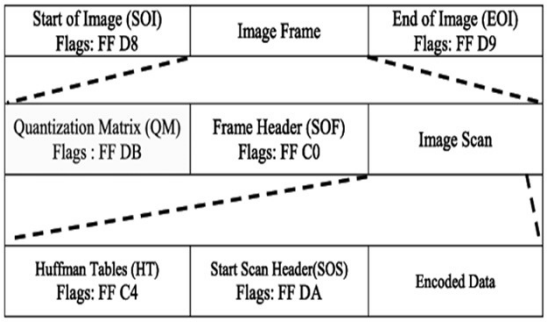
Hdhai - Copyright

14

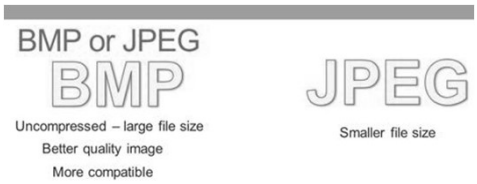
Cấu trúc ảnh số JPG/JPEG

Cấu trúc ảnh JPG / JPEG

- Một ảnh JPEG gồm một chuỗi các phân đoạn. Mỗi phân đoạn bắt đầu bằng một Marker (SOI) =1 byte 0xFF, và 1 Byte = D8
- Khung ảnh: 2 Bytes biểu thị kiểu nén (0xFFC- =DCT, 0xFFC4 =Hoffman, 0xFFDB = Quantization
- Tiếp theo: Image Scan gồm bảng mã Hoffman (nếu dùng), đánh dấu quét, dữ liệu mã hóa nén
- Kết thúc = cờ báo EOI (0xFFD9)



So sánh BMP với JPG / JPEG



JPG / JPEG dùng thuật toán LZW (Lempel-Zib-Welch) để nén ảnh

Hdhai - Copyright

15

Cấu trúc ảnh số GIF

Cấu trúc ảnh GIF (256color)

- Header = 6 bytes gồm 3 bytes Signature, 3 bytes Version

// Header

BYTE Signature[3]; */* Header Signature ("GIF") */*

BYTE Version[3]; */* GIF version("87a" or "89a") */*

// Logical Screen Descriptor

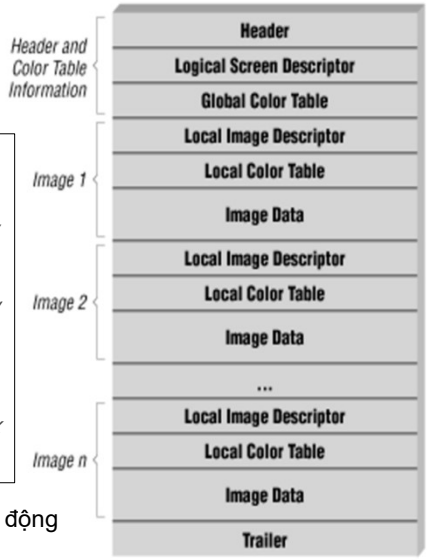
WORD ScreenWidth; */*Width of Screen in Pixels */*

WORD ScreenHeight; */*Height of Screen in Pixels*

BYTE Packed; */* Screen and Color Map Info */*

BYTE BackgroundColor; */* Background Color Index*

BYTE AspectRatio; */* Pixel Aspect Ratio */*



- Các images được quét liên tục tạo thành ảnh động

GIF dùng thuật toán LZW (Lempel-Zib-Welch) để nén ảnh

Hdhai - Copyright

16

So sánh cấu trúc ảnh số

	BMP	JPG	PNG	GIF
Compressed	FALSE	TRUE	TRUE	TRUE
Lossless	TRUE	FALSE	TRUE	TRUE
Transparency	FALSE	FALSE	TRUE	TRUE
Translucency	FALSE	FALSE	TRUE	FALSE
Recommended for photographs	FALSE	TRUE	FALSE	FALSE
Recommended for static graphics/icons	FALSE	FALSE	TRUE	FALSE
Recommended for animated graphics/icons	FALSE	FALSE	FALSE	TRUE

Hdhai - Copyright

17

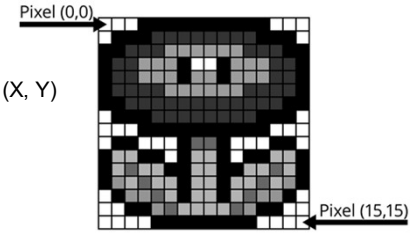
Biểu thị điểm ảnh Bitmap

Biểu thị ảnh Bitmap:

- Ma trận M x N
- Mỗi điểm của ma trận I (y, y) tương ứng 1 vị trí điểm ảnh trên mặt phẳng 2D X-Y
- Ảnh đen trắng: mỗi điểm = 8 bit (giá trị từ 0….255)
- Ảnh màu: mỗi điểm chứa RGB x 8 bit (3 x 8 =24 bit = 16 triệu màu)

Đọc điểm ảnh (Pixel):

```
import cv2
img = cv2.imread ('Image.png')
# Dùng hàm getPixel() để đọc thông tin Pixel tại vị trí (X, Y)
# Tại tọa độ X = 10, Y = 8
(b,g,r) = img[10][8]
print(b,g,r)
```



Sửa điểm ảnh (Pixel):

```
img[10][8] = (244, 244, 244)
```

Sửa kích thước ảnh:

```
half = cv2.resize(image, (0, 0), fx = 0.1, fy = 0.1)
bigger = cv2.resize(image, (1050, 1610))
stretch_near = cv2.resize(image, (780, 540), interpolation = cv2.INTER_LINEAR)
```

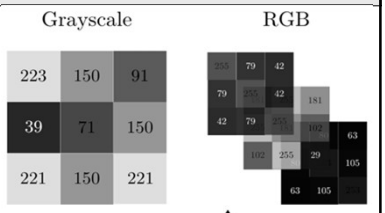
Hdhai - Copyright

18

Tạo Histogram cho ảnh số

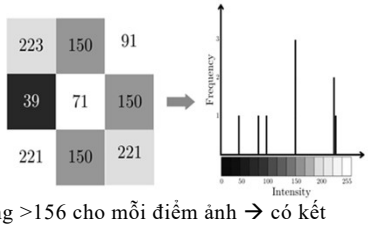
Đặc trưng của ảnh số

- Ảnh số = ma trận các điểm ảnh.
- Ảnh đen trắng: ma trận chứa các con số từ 0…255 (0 = đen, 255= trắng, các giá trị còn lại = các mức màu xám)
- Ảnh màu = 3 ma trận RGB, mỗi ma trận chứa các giá trị màu (0…255)

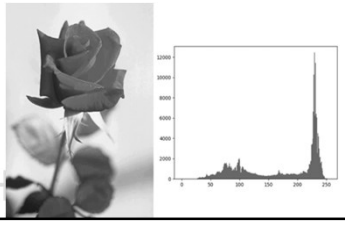


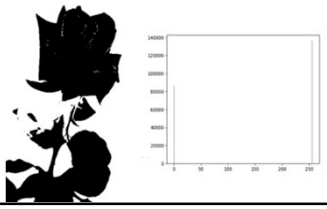
Histogram của ảnh số

- Biểu thị tần suất cho mỗi cường độ sáng (Intensity).
- Ví dụ: cường độ 150 xuất hiện ở 3 điểm ảnh (3 Pixels)
- Histogram dùng trong nhiều ứng dụng: xử lý ảnh, nhận dạng ảnh, tách ảnh,…
- Ví dụ: Histogram sau cho thấy ảnhđa phần màu sáng



Nếu đặt mức ngưỡng >156 cho mỗi điểm ảnh → có kết quả nhị phân, tách ảnh





19

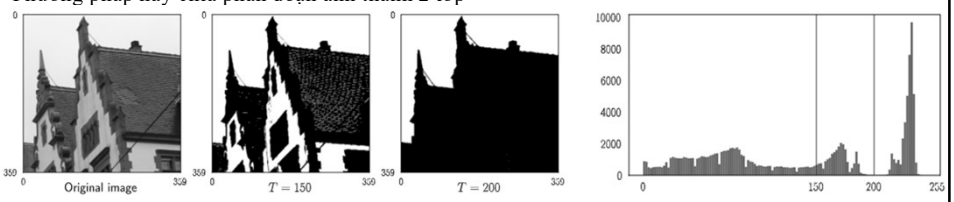
Histogram cho ảnh số - mức ngưỡng

Đặt mức ngưỡng nhị phân cho Histogram (T = giá trị ngưỡng)

Có thể gán 1 số nhị phân cho mỗi Pixel của 1 ảnh f(m,n) theo Intensity với mức ngưỡng như sau:

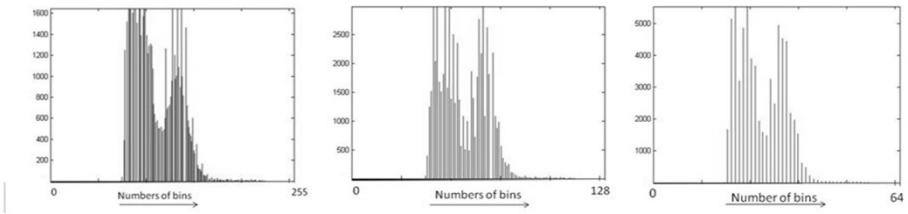
$$g(m,n) = \begin{cases} 1 & \text{if } f(m,n) \geqslant T, \\ 0 & \text{if } f(m,n) < T \end{cases}$$

Phương pháp này chia phân đoạn ảnh thành 2 lớp



Chia Bins trong Histogram

VD. Dùng 8 bit cho Pixels BW → có tới 2^8 = 256 Bins. Có thể nhóm các giá trị Intensity vào Bins



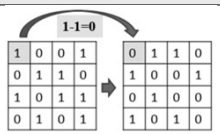
20

Histogram cho ảnh số - biến đổi cường độ, chất lượng

Chuyển đổi ảnh số qua biến đổi Intensity

Đảo ngược giá trị ảnh: $f^c(x,y) = 1 - f(x,y)$

Với 256 mức xám → $f^c(x,y) = 1 - f(x,y) / 256$

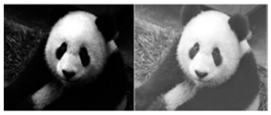


Biến đổi Gamma:

C và γ (Gamma) là hằng số: $f^c(x,y) = C * f(x,y)^{\gamma}$

Với $f(x,y)$ trong khoảng [0,1], γ lớn thì ảnh tối và ngược lại

VD: a) Gamma = 2 b) Gamma = 0,5

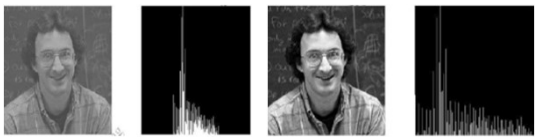


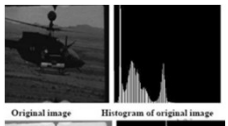
Tăng cường chất lượng ảnh (Contrast Stretch)

$$\text{Stretch}(I(r,c)) = \left[\frac{I(r,c) - I(r,c)_{\min}}{I(r,c)_{\max} - I(r,c)_{\min}}\right][MAX\text{-}MIN] + MIN.$$

$I(r,c)_{max}$, $I(r,c)_{min}$ là mức xám lớn nhất / nhỏ nhất của ảnh $I(r,c)$

MAX = 255, MIN = 0 (cho ảnh 8 bit /Pixel)



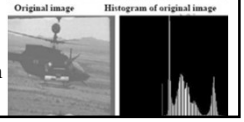


Trượt Histogram (Sliding)

$$Slide\left(I(r,c)\right) = [I(r,c)] + OFFSET$$

OFFSET là giá trị trượt:

> 0 → ảnh sáng hơn; < 0 → ảnh tối hơn



Hdhai - Copyright

21

Histogram Equalization

- Là kỹ thuật để tăng cường chất lượng ảnh (tương tự Stretch, song cho kết quả hiển thị tốt hơn)
- Mục đích: Biến đổi Histogram phẳng hơn (khác với Stretch giữ nguyên hình dạng Histogram)
- Kết quả của Histogram Equalization= nhóm các đỉnh Histogram gần nhau → điểm tối thành tối hơn, điểm sáng thành sáng hơn.

Hdhai - Copyright

22

Histogram Equalization

Các bước thực hiện: VD: ảnh 3 bit/Pixel, khoảng giá trị 0-7

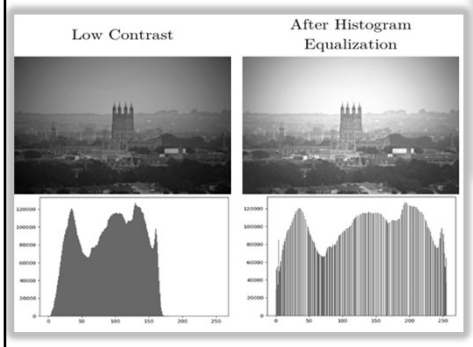
- **Bước 1:** Tính Running Sum của các giá trị Histogram.
- **Bước 2:** Chuẩn hóa (chia tổng số điểm = 51)
- **Bước 3:** Nhân với max gray level (7) và làm tròn

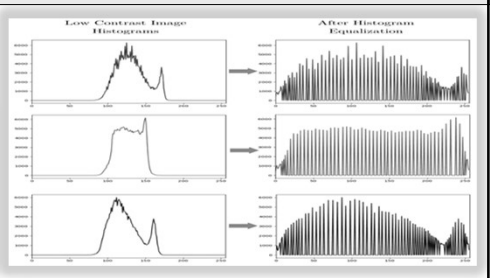
Hdhai - Copyright

23

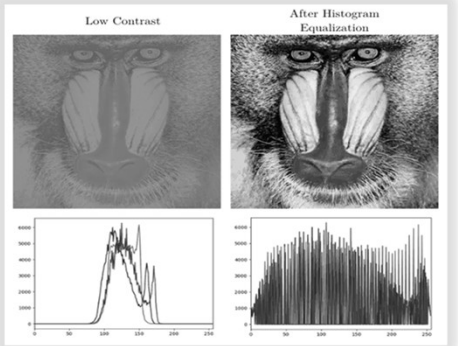
Color Image Histogram Equalization

- Có thể thực hiện cân bằng Histogram để cải thiện ảnh đối với từng màu sắc, tăng Contrast. VD: ảnh 1 không có điểm nào >175





- Cân bằng Histogram cho chất lượng ảnh tốt hơn, sắc nét hơn



Hdhai - Copyright

24

Template Matching

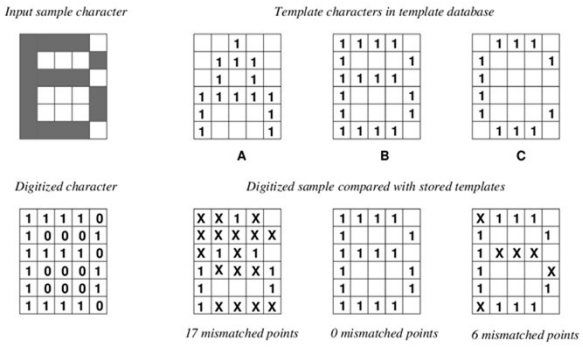
So khớp mẫu ảnh ký tự

Pattern matching

Cách 1: Truyền thống.

So sánh từng điểm (matrix or template matching). Đếm số điểm sát nhất với template

Cách 2: Biến đổi (vd. Tính moment, Fourier). Tính khoảng cách từ 1 điểm tới mọi điểm của matrix. Tạo features → So sánh features



Thuật toán để khớp mẫu hoặc nhận dạng mẫu:

- So sánh dữ liệu với các ký hiệu khác được lưu trữ dưới dạng mẫu dựa trên một bộ quy tắc đã xác định trước với các ký hiệu có tỷ lệ và phông chữ đã xác định
- Thường dùng với văn bản được nhập bằng phông chữ đã biết trước.

Hdhai - Copyright

25

Template Matching

Các thuật toán so khớp mẫu ảnh chữ viết / chữ in

1) So khớp mẫu chữ viết (Pattern Matching)

→ So sánh từng điểm (hoặc từng phần) của ảnh với 1 tập mẫu chữ có sẵn

→ Có thể đo khoảng cách giữa 2 histogram để so sánh (Chi-square distance) $\chi^2 = \sum \frac{(o-e)^2}{e}$

→ Dễ triển khai với dạng ký tự cố định, font chữ đã biết trước (VD. Ứng dụng nhận dạng số đo, biển số xe,…)

2) So khớp mẫu dựa theo HOG (Histogram of Oriented Gradients) + SVM (Support Vector Machine)

→ Tạo 2 ma trận: Gradient Magnitude (GM), Gradient Orientation (GO). GM được nhóm vào các BIN theo GO cho từng vùng cục bộ

→ Tính toán vecto đặc trưng HOG (kết hợp các vùng cục bộ tạo thành véc-tơ HOG gồm các đặc trưng ảnh)

→ VD. Ứng dụng nhận dạng chữ viết tay

Hdhai - Copyright

26

Template Matching

Phương pháp so khớp mẫu:
- Phương pháp so khớp mẫu dùng để tìm vị trí trong ảnh khớp với Template.
- Là phương pháp đơn giản nhất. Được sử dụng trong nhận dạng đối tượng, nhận dạng ảnh, nhận dạng mẫu.

Các bước thực hiện:

1) Trượt ảnh mẫu T(w,h) trên ảnh đầu vào I(W,H) theo chiều ngang (x), chiều dọc ảnh (y).
2) So sánh mỗi điểm của ảnh template với điểm ảnh của cửa sổ trượt tương ứng trên ảnh đầu vào bằng một công thức tính độ tương đồng (gọi là **Similarity Metric**). Nghĩa là mức độ tương đồng giữa hai điểm ảnh.
3) Mỗi kết quả so sánh được đưa vào **ma trận kết quả R(W-w+1,H-h+1)**. Mỗi vị trí trong ma trận R chỉ chứa Match Metric (kết quả tương đồng).
4) Tìm giá trị tối đa/hoặc tối thiểu trong ma trận R và vị trí
5) Lấy vị trí đó làm góc trên trái hình chữ nhật (w,h). Hình chữ nhật là kết quả so khớp mẫu.

Ví dụ: So khớp mẫu T với ảnh I

Hdhai - Copyright

27

Template Matching

Các công thức tính độ tương đồng (Similarity Metrics)

1) Tổng bình phương hiệu số 2 điểm ảnh (Sum of Squared Differences-SSD):
min = best match

$$R(x,y) = \sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2$$

2) Tổng bình phương sai lệch chuẩn (Normalized Sum of Squared Differences-NSSD): [0, 1]. **Min = best match**

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$$

3) Độ tương quan chéo (Cross-Correlation CC): Tổng của tích các điểm ảnh, chỉ ra vùng có tương đồng về độ tương phản (Intensity) và mẫu. **Max = best match**

$$R(x,y) = \sum_{x',y'} (T(x',y') \cdot I(x+x',y+y'))$$

4) Độ tương quan chéo chuẩn (Normalized Cross-Correlation CC): [0, 1].
Max = best match

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') \cdot I(x+x',y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$$

Hdhai - Copyright

28

Template Matching

Các công thức tính độ tương đồng (Similarity Metrics)

5) Hệ số tương quan (Correlation Coefficient CCOEFF):
Đo quan hệ tuyến tính của Template và vùng ảnh quan tâm. Trừ giá trị của điểm ảnh đi giá trị trung bình (giúp loại bỏ khác biệt độ tương phản tổng thể ảnh). Chỉ quan tâm cấu trúc của Template và vùng ảnh quan tâm. **Max = best match**

$$R(x,y) = \sum_{x',y'} (T'(x',y') \cdot I'(x+x',y+y'))$$

$$T'(x',y') = T(x',y') - 1/(w \cdot h) \cdot \sum_{x'',y''} T(x'',y'')$$
$$I'(x+x',y+y') = I(x+x',y+y') - 1/(w \cdot h) \cdot \sum_{x'',y''} I(x+x'',y+y'')$$

6) Hệ số tương quan chuẩn (Normalized Correlation Coefficient NCCOEFF): [0, 1].
Max = best match

$$R(x,y) = \frac{\sum_{x',y'} (T'(x',y') \cdot I'(x+x',y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 \cdot \sum_{x',y'} I'(x+x',y+y')^2}}$$

Hdhai - Copyright

29

Nhận dạng đặc trưng ảnh

❖Các phương pháp nhận dạng đặc trưng ảnh

Ngưỡng	Cạnh	Vùng	Histogram	Phân cụm	Học máy	Học sâu

- ❑ Ngưỡng (Thread): dựa vào giá trị màu (đơn giản nhất là đen / trắng = mức 0, 1)
- ❑ Cạnh (Edge): Phát hiện các cạnh ảnh (đường nét)
- ❑ Phân vùng (Region): Tìm các điểm ảnh tương đồng, nhóm thành các lớp
- ❑ Histogram (Histogram of Gradients = HOG): biên độ dốc và hướng biên của đối tượng ảnh
- ❑ Phân cụm (Clustering): Tìm các điểm ảnh theo đặc trưng, nhóm thành các cụm
- ❑ Học máy (Machine Learning): Trích xuất đặc trưng, phân lớp ảnh, đánh nhãn, nhận dạng đặc trưng theo nhãn
- ❑ Học sâu (Deep Learning): Sử dụng mạng nơ ron tích chập (CNN) để phân đoạn ảnh, tạo bản đồ đặc trưng, dùng hàm kích hoạt (Softmax) để phân loại nhãn

Hdhai - Copyright

30

30

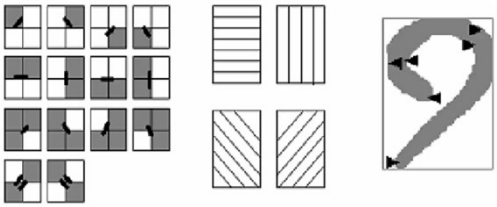
Trích chọn đặc trưng ảnh

Trích chọn đặc trưng /Tách đặc trưng (Feature extraction)

Là vấn đề khó nhất khi nhận dạng mẫu

3 nhóm kỹ thuật chính:

- Phân bố của điểm ảnh
- Biến đổi và biểu thị dạng chuỗi (Fourier, DCT, Wave)
- Phân tích cấu trúc



Thuật toán trích xuất đặc trưng:

- Chia nhỏ các ký hiệu tượng hình thành các đặc trưng cơ bản hơn như các đường góc, giao điểm hoặc đường cong để nhận dạng.
- So khớp các đặc trưng với các ký hiệu tượng hình đã lưu trữ trước đó.
- Phương pháp so khớp thường dựa vào các thuật toán học máy (điển hình là k-nearest neighbors), cho phép nhận dạng cả ảnh văn bản in và văn bản viết tay phức tạp hơn.

Hdhai - Copyright

31

31

Histogram Of Gradients (HOG)

Khái niệm:

- Là thuật toán dùng để phân đoạn ảnh (Phát hiện đối tượng)
- HOG đơn giản hơn so với các thuật toán khác như:
  + Nhận dạng ảnh: CNN (Resnet, DenseNet, Alexnet,…);
  + Phát hiện vật thể: YOLO, Fast- RCNN, Faster RCNN, Mask RCNN
- HOG tạo ra bộ mô tả đặc trưng (Feature Descriptor) để phát hiện vật thể
- Từ 1 ảnh, HOG tạo ra 2 ma trận: Gradient Magnitude (GM), Gradient Orientation (GO). GM được nhóm vào các BIN theo GO cho từng vùng cục bộ, kết hợp các vùng tạo thành véc-tơ HOG gồm các đặc trưng ảnh

Ví dụ: Phát hiện ảnh 1 người đi bộ trên hình

Hdhai - Copyright

32

Histogram Of Gradients (HOG)

Điểm cơ bản của HOG: Hình dạng của 1 vật thể cục bộ có thể được mô tả qua 2 ma trận GM (Gradient Magnitude) và GO (Gradient Orientation) /hay GD (Gradient Direction).

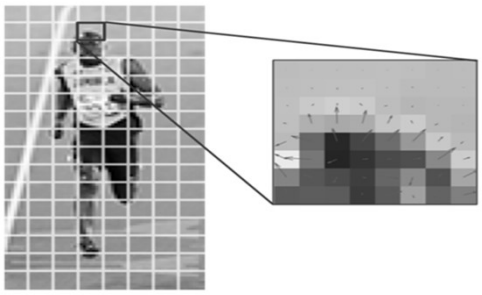
- Ví dụ: Ảnh đen trắng kích thước 22 x 16 = 352 điểm ảnh (Pixel).

  Mức xám của điểm ảnh: từ 0 … 255 (Intensity hay Gray Level), 0 = Đen, 255=Trắng

  Feature đơn giản: M = 1 x 352 matrix (1 chiều), mỗi ô có giá trị Gray Level của điểm ảnh

- Ví dụ: Ảnh màu có 3 ma trận Pixel cho R, G, B. Giá trị màu của mỗi ô từ 0 → 255 (8 bit màu).

  Feature đơn giản: M= giá trị trung bình của R, G, B tương ứng (tổng RGB / 3)



2	3	4	4	3	4	2	2
5	11	17	13	7	9	3	4
11	21	23	27	22	17	4	6
23	99	165	135	85	32	26	2
91	155	133	136	144	152	57	28
98	196	76	38	26	60	170	51
165	60	60	27	77	85	43	136
71	13	34	23	108	27	48	110

Gradient Magnitude

80	36	5	10	0	64	90	73
37	9	9	179	78	27	169	166
87	136	173	39	102	163	152	176
76	13	1	168	159	22	125	143
120	70	14	150	145	144	145	143
58	86	119	98	100	101	133	113
30	65	157	75	78	165	145	124
11	170	91	4	110	17	133	110

Gradient Direction

Hdhai

33

Histogram Of Gradients (HOG)

- **Feature Descriptor:** Bộ mô tả đặc trưng, biến đổi dữ liệu thành các đặc trưng để phân loại hoặc nhận diện vật thể.
- **Histogram:** biểu đồ Histogram biểu diễn phân phối cường độ màu sắc theo khoảng giá trị.
- **Gradient:** đạo hàm của véc tơ cường độ màu sắc phát hiện hướng đi của vật thể trong hình
- **Local cell:** Ô cục bộ (Cell). 1 ảnh chia ra các cells bởi 1 lưới ô vuông. Mỗi cell là 1 ô cục bộ.
- **Local portion:** Vùng cục bộ (Block). 1 vùng trích xuất từ ô vuông trên ảnh. GM và GO sẽ được tính cho mỗi vùng cục bộ.
- **Local normalization:** Chuẩn hóa trên 1 vùng cục bộ để đồng nhất các giá trị cường độ màu sắc về chung một phân phối.
- **Gradient direction (GO):** Hướng gradient. Độ lớn góc giữa véc tơ gradients x và y giúp xác định hướng thay đổi cường độ màu sắc. Gx, Gy theo hướng x và y của ảnh.

$$\theta = \text{acrtan}(\frac{G_y}{G_x})$$

- **Gradient Magnitude (GM):** Độ lớn gradient. Chiều dài của véc tơ gradient theo hướng x và hướng y. Biểu diễn phân phối histogram hướng gradient (theo các bins) sẽ cho kết quả véc tơ mô tả đặc trưng HOG

$$|G| = \sqrt{G_x^2 + G_y^2}$$

Hdhai - Copyright

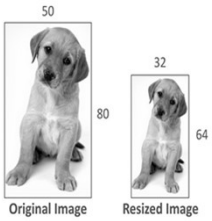
34

Histogram Of Gradients (HOG)

Các bước thực hiện HOG:

1) Bước 1: Tiền xử lý

- Chuẩn lại màu, Gamma (độ nét, cường độ)
- Chuẩn hóa kích thước theo tỷ lệ 1: 2
  ví dụ đưa ảnh về 32 x 64 hoặc 64 x 128,…

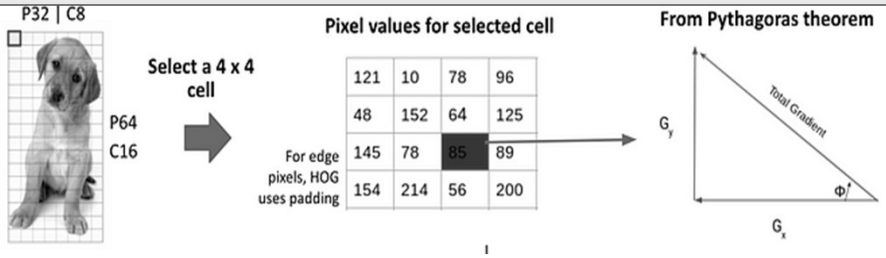


- Resized image size: 32x64
- HOG Cell size: 4x4
- Recommended:
  - Image size: 64x128
  - cell size: 8x8 / 16x16

Hdhai - Copyright

35

Histogram Of Gradients (HOG)



2) Bước 2: Tính GM và GO

- Gx là Gradient theo hướng x (Gx = giá trị khác biệt giữa trái và phải)

  Ví dụ trên hình, tính Gx cho ô màu đỏ 85: Trái = 78, phải = 89 → Gx = 89 – 78 = 11

- Gy là Gradient theo hướng y (Gy = giá trị khác biệt giữa trên và dưới)

  Ví dụ trên hình, tính Gy cho ô màu đỏ 85: Trên = 64, Dưới = 56 → Gy = 64 – 56 = 8

→ Theo Pitago tính GM, GO (Chú ý: GO từ 0 – 180°)

$GM = |G| = \sqrt{G_x^2 + G_y^2} = \sqrt{11^2 + 8^2} = \sqrt{185} = 13{,}6$ *GO = arctan2(G_y / G_x) = 36*

Hdhai - Copyright

36

Histogram Of Gradients (HOG)

3) Bước 3: Tính HOG

GO có giá trị từ 0 … 180 độ → HOG chọn 20 độ cho mỗi Bins → Có 9 Bins.
Bin1: 20, Bin2: 40, … Bin9: 160. Điền GM theo các BIN của GO như sau:

Total Gradient Magnitude = 13.6
Orientation (Φ) = 36 deg.

(40-36)/20 (36 - 20)/20

GM

Magnitude		(4/20)*13.6	(16/20)*13.6						
Bin	0	20	40	60	80	100	120	140	160
Features	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**

Angle Weight 16 4
20 36 40

Matrix 1 x 9

Có thể dùng công thức phân chia theo Interpolation:

$$BIN\ \ l\text{-}1 \quad x_{l-1} = \frac{(x_1 - x)}{x_1 - x_0} * y \qquad BIN\ \ l \quad x_l = \frac{(x - x_0)}{x_1 - x_0} * y$$

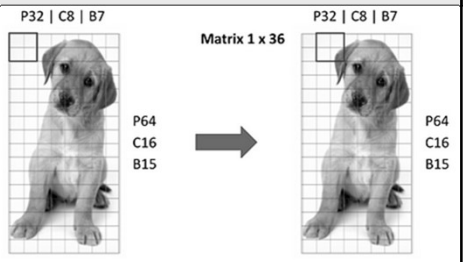
Hdhai - Copyright

37

Histogram Of Gradients (HOG)

4) Bước 4: Chuẩn hóa

- Do các gradient cục bộ nhạy cảm với toàn ảnh → cần giảm biến thiên bằng cách chuẩn hóa Gradient.
= Gộp 4 Cells thành Blocks, dịch chuyển mỗi lần 1 Cell (xem hình) → có 7 block theo chiều X, 15 block theo chiều Y.
→ Với ma trận feature 1x9 mỗi Cell → Mỗi block (4 Cell) sẽ có 1 ma trận chuẩn 1x36.



Sau khi có HOG features → Nhận dạng ảnh / phát hiện vật thể …

Hdhai - Copyright

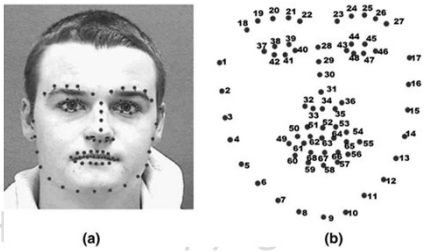
38

Nhận dạng đặc trưng ảnh

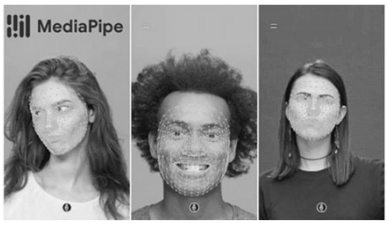
❖ **Nhận dạng đặc trưng khuôn mặt**

- Định vị khuôn mặt (face localization): xác định vị trí ảnh của khuôn mặt đơn (tách nhiều khuôn mặt)
- Phát hiện đặc trưng khuôn mặt (facial feature detection): phát hiện đặc tính mắt, mũi, miệng, môi, tai, lông mày, lông mi, màu (màu da, màu mắt, màu tóc,…)
- Phát hiện trạng thái khuôn mặt (face behaviour): vui, buồn, giận, đau, cau có,…
- Nhận dạng mặt (face identification): so sánh ảnh input với cơ sở dữ liệu, so khớp ảnh
- Kiểm chứng/xác thực khuôn mặt (face verification / authentication): kiểm tra, xác thực với ảnh đầu vào

Các tập Dataset khuôn mặt chuẩn (2D):
68 tọa độ x và y = các điểm đặc trưng nhất của một khuôn mặt người.



Các tập Dataset khuôn mặt chuẩn (3D): 468 tọa độ x và y = các điểm đặc trưng nhất của một khuôn mặt người.



39

Nhận dạng đặc trưng ảnh

❖ **Ví dụ về nhận dạng khuôn mặt**

- 1) Bước 1: Tạo mô hình (tạo tập huấn luyện – Training dataset từ thử viện có sẵn ? → OpenCV

```
import PIL.Image
import PIL.ImageDraw
import face_recognition   ### Cần cài pip install face_recognition
image=face_recognition.load_image_file("sample_image.jpg")
```

- 2) Bước 2: Sử dụng bộ phân loại (Sliding Window Classifier)
  + Chuyển đổi ảnh → đen /trắng
  + Sử dụng HOG (Histogram of Gradients) → tìm vị trí khuôn mặt
- 3) Bước 3: Huấn luyện mô hình → Sử dụng HOG
- 4) Bước 4: Phát hiện
  + Tải ảnh, chuyển vào bộ nhớ
  + Phát hiện khuôn mặt trong ảnh

```
face_locations = face_recognition.face_locations(image)
pil_image = PIL.Image.fromarray(image)  # convert ảnh → PIL
for face_location in face_locations:
   top,right,bottom,left =face_location
   draw_shape = PIL.ImageDraw.Draw(pil_image)
   draw_shape.rectangle([left, top, right, bottom],outline="red")

pil_image.save("images/output_image.jpg")
pil_image.show()
```

Hdhai - Copyright

40

40

Nhận dạng vân tay (Fingerprint)

- **Fingerprints**
  - Khả năng nhận dạng cá nhân chính xác. Đã ứng dụng trong lĩnh vực truy tìm tội phạm. Các đặc trưng khác có thể đổi, song Fingerprints thì không.
  - Ở thời cổ đại, tại thành Babilon đã tìm thấy dấu vân tay của nô lệ
  - Thế kỷ 14: Fingerprints đã dùng cho công chức chính phủ
  - 1686: GS. Malpighi (ĐH Bologna, Italy) phát kiến các đỉnh, đường xoắn, các vòng ở vân tay (chưa sử dụng).
  - 1823: GS. Prukinje (ĐH Breslau, Polen) tìm ra 9 mẫu vân tay (chưa sử dụng)
  - 1856: Herschel (quận trưởng Hooghly, India) sử dụng vân tay cho các hợp đồng dân sự. Khẳng định: các vân tay là duy nhất cho mỗi cá nhân, giữ nguyên suốt cuộc đời họ
  - 1880: Bác sỹ Faulds (Bệnh viện Tokyo, Japan) chính thức sử dụng Fingerprints cho nhận dạng bệnh nhân
  - 1883: Đại văn hào Mark Twain đã viết 2 cuốn sách "Life on Mississippi" và Pudd'n head Wilson", trong đó mô tả một tòa xử án kết tội kẻ giết người bằng nhận dạng dấu vân tay
  - 1892: Vụ án đầu tiên sử dụng Fingerprints để định danh cá nhân tại Cảnh sát Argentina
  - 1903: Hệ thống nhận dạng vân tay đầu tiên tại US
  - 1918: Phát hiện ra 12 chi tiết của vân tay cho kết quả nhận dạng đúng
  - 1946: FBI sử dụng 100 triệu thẻ vân tay cá nhân (phát hiện 25-30 triệu trường hợp tội phạm)
  - 2005: FBI đã định danh 49 triệu tội phạm bằng nhận dạng vân tay

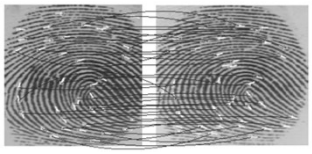
Hdhai - Copyright

41

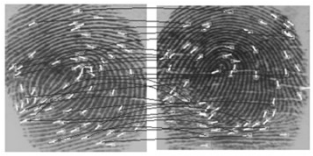
41

Nhận dạng vân tay (Fingerprint)

- **Nhận dạng vân tay**

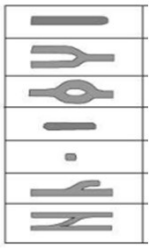


- Nhận dạng 2 vân tay giống nhau 43 chi tiết (phải). 39 chi tiết (trái)

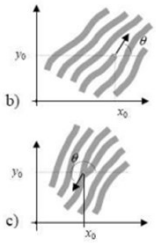


- Nhận dạng 2 vân tay khác nhau 65 chi tiết (phải). 64 chi tiết (trái) 25 sai khác giữa 2 mẫu vân tay

- **Các chi tiết vân tay dùng để xác thực**



- nét đứt (termination)
- rẽ nhánh (Bifurcation)
- tạo hồ (Lake)
- sườn độc lập (Independent ridge)
- điểm, đảo (Point, island)
- dấu móc (Spur)
- đan chéo (Crossover)



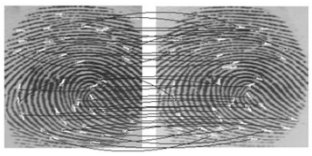
Hdhai - Copyright

42

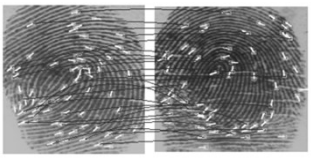
42

Nhận dạng vân tay (Fingerprint)

❑ **Nhận dạng vân tay**

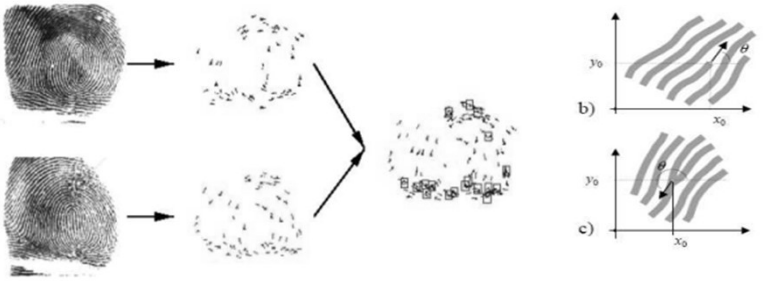


▪ Nhận dạng 2 vân tay giống nhau 43 chi tiết (phải). 39 chi tiết (trái)



▪ Nhận dạng 2 vân tay khác nhau 65 chi tiết (phải). 64 chi tiết (trái) 25 sai khác giữa 2 mẫu vân tay

❖ **So khớp vân tay**



Hdhai - Copyright

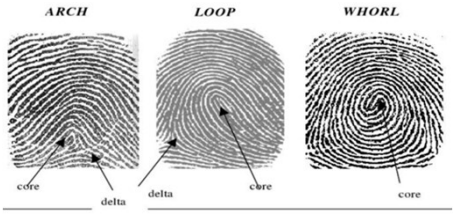
43

43

Nhận dạng vân tay (Fingerprint)

❖ **Mẫu vân tay**

- Phân loại Henry: 03 mẫu cơ bản
  - Hình vòm (Arches)
  - Hình vòng tròn (Whorls)
  - Hình vòng lặp (Loop)



❖ **Cách tính mẫu vân tay để so khớp**

$$\text{Hệ số C} = \frac{(Ri + Rr + Lt + Lm + Lp + 1)}{(Rt + Rm + Rp + Li + Lr + 1)}$$

Hệ số 16 8 4 2 1

Nếu có vòng tròn → 1 *(Whorls=1)*, nếu không có → 0 *(Loops và Arches = 0)*

R = Right, L = Left,
t= ngón cái, i= ngón trỏ, m=ngón giữa, r= ngón đeo nhẫn, p= ngón út

Hdhai - Copyright

44